```
makeCompetition = ({max, sort}) ->
  find = (name) ->
    document.querySelector ".#{name}"

  color = (element, color) ->
    element.style.background = color

  insert = (teams...) ->
    root = document.querySelector '.teams'
    for team in teams
      element = document.createElement 'li'
      element.innerHTML = "#{team.name} (#{team.points})"
      element.className = team.name
      root.appendChild element

  highlight = (first, rest...) ->
    color find(first.name), 'gold'
    for team in rest
      color find(team.name), 'blue'

  rank = (unranked) ->
    unranked.sort(sort).slice(0, max)

  initialize = (unranked) ->
    ranked = rank unranked

    insert ranked...
    first = ranked.slice(0, 1)[0]
    rest = ranked.slice 1
    highlight first, rest...

  { initialize }

sortOnPoints = (a, b) ->
  a.points > b.points

window.onload = ->
  competition = makeCompetition(max: 5, sort: sortOnPoints)
  competition.initialize [
    { name: 'wolverines', points: 22 }
    { name: 'wildcats', points: 11 }
    { name: 'mongooses', points: 33 }
    { name: 'raccoons', points: 12 }
    { name: 'badgers', points: 19 }
    { name: 'baboons', points: 16 }
  ]
```

A maker function that returns a competition object. The arguments are named by using object syntax in the function declaration.

Rank the competitors using the sort method on the array prototype.

Return an object with an initialize property and the value of the initialize variable as the value. This is the shorthand syntax for {initialize: initialize}.

The sorting strategy for the teams that will be given to the competition; it sorts on points.

Pass in object as named parameters to the maker function.

CoffeeScript mixes syntax and semantics from JavaScript (both current and future versions) as well as from other programming languages. Some things (such as destructuring) are very useful, whereas other things are (or can be) less useful. Take `null`, for example.

7.3. No nulls